

Methodology

How `logjam` turns raw port-activity data into bottleneck, recovery, and opportunity signals. This document is the reference for anyone reviewing a finding the tool produced.

Repository: <https://github.com/ubeast/logjam>

1. Data source

Everything in v1 comes from **IMF PortWatch**, a free public platform run by the IMF with the UN Global Platform. PortWatch estimates daily maritime activity from satellite AIS (Automatic Identification System) signals on ~90,000 ships, published as two ArcGIS feature layers:

LAYER	GRAIN	FIELDS USED
<code>Daily_Ports_Data</code>	one row per port per day, ~2,065 ports	<code>portcalls_*</code> (arrivals by cargo class), <code>import_*</code> / <code>export_*</code> (modelled trade-volume estimates)
<code>Daily_Chokepoints_Data</code>	one row per chokepoint per day, 28 chokepoints	<code>n_*</code> (transit counts by cargo class), <code>capacity_*</code> (estimated aggregate cargo capacity of transiting vessels)

Cargo classes: `container`, `dry_bulk`, `general_cargo`, `roro`, `tanker`, plus `cargo` (all dry cargo) and `total`.

Refresh cadence. PortWatch updates weekly, Tuesdays ~09:00 ET. It revises the most recent ~2 weeks upward as more AIS is processed, so the newest days always under-report. The tool re-pulls a 10-day overlap on every incremental refresh, and analyses that need a stable "current" value exclude the trailing days (`recovery_trailing_exclude_days`, default 2).

What the data does and does not measure

MEASURES	DOES NOT MEASURE
How many vessels arrived at a port / crossed a chokepoint line, per day	Time a vessel spent waiting at anchor (queue length)
Estimated aggregate cargo capacity of those vessels	Berth dwell time
Modelled import/export trade volume	Actual TEU / tonnage loaded
	Voyage duration (e.g. Suez vs Cape routing)

A chokepoint transit is an instantaneous line-crossing count, not a transit *time* — the Strait of Hormuz itself is a few hours' passage.

1a. Live AIS (AISStream.io)

The queue signal PortWatch lacks comes from **AISStream.io**, a free WebSocket feed of raw AIS position reports (needs a free API key). It is a *push* stream, not a batch endpoint, so the tool **samples** it: connect for a fixed number of minutes (`ais_sample_minutes` , default 30), capture every position report inside the subscription regions, disconnect, and append the raw rows to date-partitioned Parquet under `data/ais_raw/` (git-ignored). A separate step reduces one day of captures to per-zone daily metrics.

Geofences (`resources/ais_zones.yaml`): a circular anchorage catchment (`port_radius_km` , default 25 km) around each watch-listed port, and a square box (`chokepoint_half_deg` , default $\pm 0.35^\circ$) around every PortWatch chokepoint that falls inside a subscription region. Port and chokepoint centres come from PortWatch's own coordinate table (`resources/geo/portwatch_places.json`).

Derived metrics (source `aisstream` , one value per zone per day):

METRIC	ZONE	DEFINITION
<code>vessels_at_anchor</code>	port	distinct MMSIs whose median speed in the catchment is $\leq$ <code>ais_anchor_max_sog_kn</code> (1 kn) or which reported AIS nav-status 1/5 (anchored/moored)
<code>vessels_moving</code>	port	distinct MMSIs under way (median SOG $\geq$ 3 kn) in the catchment
<code>ais_transiting</code>	chokepoint	distinct MMSIs under way inside the chokepoint box during the sample

These are counts *observed during the sample window*, not full-day censuses. They are comparable day to day because the sampling cadence is fixed, and a day with fewer than `ais_min_messages_per_day` (200) raw messages is skipped as too thin. `vessels_at_anchor` is a standing quantity (a queue), so a snapshot measures it directly; `ais_transiting` is a rate proxy and a same-day cross-check on PortWatch's `n_total` (which lags a week and reads exact-zero on missing data).

2. Pipeline

```
ingest (PortWatch ArcGIS, paged)                                ingest/portwatch.py
-> tidy long frame: entity | date | metric | value
-> DuckDB `observation` table (idempotent upsert)  store/

sample (AISStream WebSocket, time-boxed)                       ingest/aisstream.py
-> raw Parquet captures under data/ais_raw/
-> reduce to per-zone daily metrics                    analytics/ais_reduce.py
-> same `observation` table (source 'aisstream')

-> per-series baselines                                analytics/baseline.py
-> bottleneck + anchor-queue signals                  analytics/detect.py
```

-> opportunity signals	analytics/opportunity.py
-> recovery status (on demand)	analytics/recovery.py

`logjam refresh` runs the PortWatch pull, folds in any un-reduced AIS captures, and recomputes everything.
`logjam ais-collect` runs the sampler. Both are idempotent — safe to re-run.

3. Baselines

Every `(entity, metric)` pair is a daily time series. The tool builds **two** baselines for each, because they answer different questions.

3a. Short baseline — *is something breaking right now?*

A trailing **robust** band over `baseline_window_days` (default 56):

- `expected` = trailing median of the series
- `scale` = trailing MAD (median absolute deviation) $\times 1.4826$, which rescales MAD to be comparable to a standard deviation
- `robust_z = (value - expected) / scale`

Median and MAD rather than mean and standard deviation because port-call counts are spiky: a single storm day or a data gap should not widen the band for weeks. The window is **causal** (`closed="left"`) — the current day is excluded from its own baseline, so a genuine anomaly cannot dampen its own signal. Series with fewer than `baseline_min_observations` (default 21) points in the window are skipped.

Scale note: the strict definition of MAD re-centres each rolling window on that window's own median. The tool uses a fast two-pass approximation — the trailing median of absolute deviations from the trailing median — which is $\sim 100\times$ faster across tens of thousands of series and produces a materially identical band.

3b. Year-over-year baseline — *is a known disruption still ongoing?*

The short baseline has a blind spot: once an anomaly outlives the 56-day window, the "expected" value decays toward the new abnormal level and `robust_z` returns to ~ 0 . A six-month blockade looks normal because the blockade *is* the recent history.

The YoY baseline compares each day to the same calendar period one year earlier:

- `expected_yoy` = median of observed values in a $\pm$ `yoy_halfwidth_days` (default 14) window centred `yoy_lag_days` (default 365) before the date
- `robust_z_yoy = (value - expected_yoy) / scale` (short-baseline `scale` as the yardstick)
- `pct_of_yoy = value / expected_yoy`

Computed in one SQL self-join. NULL until a series has ~1 year of prior history, so `initial_backfill_days` (default 900) must comfortably exceed `yoy_lag_days`.

Limitation. YoY answers "vs a year ago," not "vs pre-disruption." If a disruption is more than a year old, both the current and the year-ago figure are depressed and `pct_of_yoy` drifts back toward 1.0 — the tool would report "recovered" when it means "stably degraded." For an event under a year old the comparison is clean. A fixed pre-crisis reference window is on the roadmap; the report generator (§6) uses one explicitly for exactly this reason.

4. Bottleneck detection

`analytics/detect.py`. Every PortWatch metric is "more = more flow" (port calls, transits, trade volume, cargo capacity), so a bottleneck is always the **negative tail**. A day is flagged when **either**:

- `robust_z ≤ -bottleneck_z_threshold` (default 2.0) with `≥ baseline_min_observations` support — a fresh onset; or
- `robust_z_yoy ≤ -bottleneck_z_threshold` with `≥ yoy_min_observations` support — still materially below a year ago

The signal row records `value`, the `expected` it was measured against, the z-score, `severity = |z|` for ranking, and a `detail.trigger` of `short`, `yoy`, or `both`. When both fire, the row reports whichever baseline is more extreme.

Anchor-queue spikes are the mirror image: for `vessels_at_anchor` (AIS), *more* means flow is blocked, so `detect_congestion` flags the **positive** short-baseline tail (`robust_z ≥ +bottleneck_z_threshold`) as a bottleneck signal with `detail.trigger = "ais_congestion"`. AIS `vessels_moving` and `ais_transiting` are throughput-like and use the ordinary negative-tail rule above.

5. Recovery status

`analytics/recovery.py`, surfaced as `logjam recovery -s <name>`. For each `(entity, metric)` matching the name search it computes, over a trailing `recovery_window_days` (default 7) window ending `recovery_trailing_exclude_days` (default 2) before the last available date:

- `value` — median throughput in that window
- `expected_yoy`, `pct_of_yoy` — median over the window (§3b)
- `pct_of_yoy_prior` — same window shifted 28 days back, for the trend column

Verdict (`pct` = `pct_of_yoy`):

VERDICT	CONDITION
<code>recovered</code>	<code>pct ≥ 1 - recovery_tolerance</code> (default ≥ 0.80)
<code>partial recovery</code>	<code>0.75 ≤ pct < 0.80</code>
<code>disruption ongoing</code>	<code>0.40 ≤ pct < 0.75</code>
<code>severe disruption ongoing</code>	<code>pct < 0.40</code>
<code>insufficient year-ago history</code>	<code>expected_yoy</code> is NULL

Trend: `improving` / `worsening` if `pct - pct_prior` exceeds ± 0.05 , else `flat`.

Count vs. capacity

A fall in vessel *count* is ambiguous on its own — it could mean fewer, larger ships moving the same cargo. Cross-check with the `capacity_*` metric:

- count **and** capacity both down proportionally → real throughput collapse
- count down, capacity flat → fleet-composition shift, cargo throughput largely maintained
- capacity down harder than count → the remaining vessels are *smaller* than normal

And check whether the fall is **uniform across cargo classes** (route-wide blockage) or **concentrated** in one class (commodity-specific restriction / sanction).

6. Opportunity detection

`analytics/opportunity.py`. `resources/substitution_groups.yaml` defines sets of ports that can realistically absorb each other's volume on a trade lane. For each group and date, if at least one member is bottlenecked (`robust_z ≤ -bottleneck_z_threshold`) and another is running at or above its own baseline (`robust_z ≥ -opportunity_z_threshold`, default -1.5), the tool emits an opportunity signal for the alternative, ranked by how severe the congested side is. `detail` carries the congested port, its drop, and the alternative's lift.

7. Report generators

`scripts/reports/*.py` produce the **disruption briefs** in `reports/`. Each generator queries the local DuckDB store, writes every cited figure to `reports/data/<slug>.json`, and renders `<slug>.md` + `<slug>.html` through the shared `scripts/reports/_brief.py` (dataclasses + one Markdown/HTML renderer + the shared visual system and SVG chart library). `scripts/build_pdfs.py` renders each `.html` and this document to PDF.

BRIEF	CHOKEPOINT	EVENT	BASELINE
<code>hormuz_container_2026</code>	Strait of Hormuz (6)	2026 Iran conflict	Sep–Nov 2023 (reroute §: Sep 2025 – Feb 2026)
<code>suez_redsea_2026</code>	Suez Canal (1)	2023 Red Sea / Houthi crisis	Sep–Nov 2023
<code>horn_of_africa_2026</code>	Bab el-Mandeb (4)	2023 Red Sea / Houthi crisis	Sep–Nov 2023

All three use an explicit **fixed pre-crisis window** rather than the rolling YoY baseline (§3b), because by 2026 a trailing baseline treats the disrupted level as normal. "Current" figures use the most recent settled month (PortWatch revises its last ~2 weeks upward). The briefs need history back to 2023 —

`LOGJAM_INITIAL_BACKFILL_DAYS=1400 uv run logjam refresh --full`. Re-run a generator after any `logjam refresh` to regenerate its figures.

8. Known limitations

1. **AIS is sampled, not continuous.** `vessels_at_anchor` / `ais_transiting` count what was in the zone during a ~30-minute window, not a full day. A queue (standing quantity) survives this well; a transit *rate* is only a proxy. Terrestrial AIS coverage is strong near ports and thin far offshore, and the Persian Gulf has known GPS interference. Berth-dwell time still needs per-vessel state tracking (roadmap).
2. **YoY ≠ pre-crisis** for disruptions over a year old (§3b).
3. **Trailing-day under-reporting.** The newest ~2 weeks of PortWatch data read low; `bottlenecks` does not yet exclude them (only `recovery` does), so the freshest bottleneck rows can be data artifacts, not real stoppages.
4. **Estimates, not measurements.** `import_*` / `export_*` are PortWatch models; `capacity_*` is derived from vessel particulars, not manifests. Directional, not exact.
5. **Substitution groups are hand-curated.** The reroute analysis only sees ports listed in `substitution_groups.yaml`.